

Stack :-

A stack or LIFO is an abstract data type that serves as a collection of elements with two principal operations: push which add an element to the collection and pop which remove the last element that was added.

In stack both operations of push and pop takes place at the same end that is top of the stack.

It can be implemented by using both array & Linked list.

Insertion : $O(1)$

Deletion : $O(1)$

Access time : $O(n)$ [worst case]

Insertion & Deletion are allowed ^{on} one end.

Example :-

- Stacks are used for maintaining funⁿ calls (last called funⁿ must finish execution first) we can always remove recursion with the help of stacks.

- Stacks are also in ~~cases~~ used in cases where we have to reverse a word Check for balanced parenthesis and in editors where the word you typed the last is the first to be removed when you use undo operation. To similarly to implement back functionality in web browser.

Stack by using Array

```
public class Stack {  
    int array[] = null;  
    int capacity = 5;  
    int top = -1;  
  
    public Stack ()  
    {  
        this.array = new int [capacity];  
    }  
  
    public Stack (int capacity)  
    {  
        this.capacity arr = new int [capacity];  
        this.capacity = capacity;  
    }  
  
    public boolean isFull ()  
    {  
        return (top+1) == capacity;  
    }  
  
    public boolean isEmpty ()  
    {  
        return top == -1;  
    }  
}
```

```
    public void push (int data)  
    {  
        if (isFull())  
            System.out.println ("stack is full can't insert")  
        else  
            return;  
        array[++top] = data;  
        System.out.println (data + " item pushed");  
    }  
  
    public void pop ()  
    {  
        if (isEmpty())  
            return;  
        System.out.println ("stack is underflow");  
        int data = array[top--];  
        System.out.println (data + " item popped")  
    }  
  
    public int peek ()  
    {  
        if (isEmpty())  
            return -1;  
        return array [top];  
    }  
}
```

```

public static void main (String args []) {
    Stack st = new Stack ();
}

```

```

st.push (3);
st.push (4);
st.push (9);
st.push (5);
st.push (7);

```

```

System.out.println ("Before removing");
System.out.println ("isFull: " + st.isFull());
System.out.println ("isEmpty: " + st.isEmpty());

```

```

// After removing
st.pop();
st.pop();
st.pop();
st.pop();
st.pop();

```

```

System.out.println ("isFull: " + st.isFull());
System.out.println ("isEmpty: " + st.isEmpty());

```

```

}

```

Pros:- Easy to implement. Memory is saved as pointers are not

Cons:- It is not dynamic. It doesn't grow or shrink depending on needs at the time.

Stack implementation using Linked List.

```

public class StackLL {

```

```

    private ListNode top;
    private int length;

```

```

    private class ListNode {

```

```

        private int data;
        private ListNode next;
    }

```

```

    public StackLL (int data) {

```

```

        this.data = data;
    }

```

```

    public StackLL () {

```

```

        length = 0;
        top = null;
    }

```

```

}

```

```

    { public int length ()
      { return length;
      }

      public boolean isEmpty ()
      { return length == 0;
      }

      public void push (int data)
      { ListNode temp = new ListNode (data);
        temp.next = top;
        top = temp;
        length++;
        System.out.println (data);
      }

      public int pop ()
      {
        if (isEmpty ())
        { throw new EmptyStackException ();
        }
        int result = top.data;
        top = top.next;
        length--;
        return result;
      }
    }

```

```

    { public int peek ()
      { if (isEmpty ())
        { throw new EmptyStackException ();
        }
        return top.data;
      }
    }

```

```

    { public static void main (String args [])
      { StackLL stack = new StackLL ();

        stack.push (9);
        stack.push (5);
        stack.push (4);

        System.out.println (stack.peak ());
        stack.pop ();
        System.out.println (stack.peak ());
        stack.pop ();
        System.out.println (stack.peak ());
      }
    }

```

Pros: The linkedlist implementation of stack can grow and shrink according to the needs at runtime.

Cons: Requires extra memory due to involvement of pointers.

```

import java.util.Scanner;
# Reverse String
import java.util.Stack;
public class ReverseString {
    public static String reverse (String str)
    {
        Stack < Character > stack = new Stack <>();
        Char [] Chars = str.toCharArray();
        for (Char ch : Chars )
        {
            stack.push(ch);
        }
        for (int i=0; i < str.length; i++)
        {
            Chars[i] = stack.pop();
        }
        return new String (Chars);
    }
}

public static void main (String [] args)
{
    ReverseString st = new ReverseString ();
    Scanner sc = new Scanner (System.in);
    String str = sc.nextLine ();
    System.out.println (st.reverse (str));
}

```

```

# Infix to Postfix
import java.util.Scanner;
import java.util.Stack;

public class InfixToPostfix {
    static int preP {
        Switch (ch) {
            case '+':
                return -1;
            case '*':
                return -1;
            case '/':
                return -2;
            case '^':
                return -3;
        }
        return 1
    }

    static String infixToPostfix (String str)
    {
        String result = new String ("");
        for (int i=0; i < str.length(); i++)
        {
            Char ch = str.charAt (i);
            if (Character.isLetterOrDigit (ch))
                Character.isLetterOrDigit (ch)
                result += ch;
        }
    }
}

```

```
else if (ch == 'c')
    stack.push(c);
```

```
stack.push(c);
```

```
else if (ch == ')') {
    while (!stack.isEmpty() &&
           stack.peek() != '(')
        result += stack.pop();
```

```
result += stack.pop();
```

```
stack.pop();
```

```
}
```

```
else
```

```
{
```

```
while (!stack.isEmpty() &&
       prep(ch) <= prep(stack.peek()))
```

```
{
    result += stack.pop();
```

```
}
```

```
stack.push(ch);
```

```
}
```

```
}
```

```
while (!stack.isEmpty())
```

```
{
    if (stack.peek() != '(')
```

```
return "Invalid Expression";
```

```
result += stack.pop();
```

```
return result;
```

```
{ public static void main (String args[])
```

```
{ Scanner sc = new Scanner (System.in);
```

```
System.out.print ("Enter your Equation");
```

```
String str = sc.nextLine();
```

```
System.out.print ("Postfix Exp: ") + infixToPostfix (str);
```

```
}
```

```
#
```

Checked For Balanced Brackets. in an expression (well formedness) using stack.

- Declare a Character stack s.

- Now traverse the expression String exp.

1. If the current character is open bracket ('(' or '{' or '[') then push in to stack.

2. If the current character is closing bracket (')' or '}' or ']') then pop from stack

and if the popped character is the matching starting bracket then fine, else brackets are not balanced.

```

import java.util.*;

public class BalancedBrackets {

    static boolean areBalancedBracket (String exp) {
        Deque <Character> stack = new ArrayDeque
        <Character> ();

        for (int i=0; i < exp.length(); i++) {
            char x = exp.charAt(i);

            if (x == '(' || x == '{' || x == '[') {
                stack.push(x);
                continue;
            }

            if (stack.isEmpty())
                return false;

            char check;

            switch (x) {
                case ')':
                    check = stack.pop();
                    if (check == '{' || check == '[')
                        return false;
                    break;

```

```

                case '}':
                    check = stack.pop();
                    if (check == '(' || check == '[')
                        return false;
                    break;

                case ']':
                    check = stack.pop();
                    if (check == '{' || check == '(')
                        return false;
                    break;
            }

            return (stack.isEmpty());
        }

        public static void main (String args[]) {
            String exp = "{[[]]}";

            if (areBracketBalanced (exp))
                syso ("Balanced");
            else
                syso ("Not Balanced");
        }
    }

    Time Complexity O(n)
    Auxiliary space O(n) for stack.

```

Application of stack:-

- 1) Balancing of symbols conversion.
- 2) infix to postfix / prefix feature in web browsers.
- 3) Forward Backward like topological sorting.
- 4) In Graph Algorithm like connected components and strongly connected any modern computer and memory management for a running program.
- 5) In memory management that is running in uses as the primary program that is running in purpose. Each program has its own memory allocation.
- 6) String reversal is also a another application of stack. Here one by one each character get inserted into the stack. So the first character of string is on the bottom of the stack and the last element of string is on the top of stack. After performing the pop operations on stack we get string reverse order.

Reverse Stack using recursion :-

```
import java.util.*; Stack;
```

```
class Test{
```

```
    static Stack < Character > st = new Stack<>()
```

```
    {
        static void insert_at_bottom (char x)
```

```
        {
            if (st.isEmpty())
                st.push(x);
```

```
        }
        else
```

```
        {
            char a = st.peek();
            st.pop();
            insert_at_bottom (x);
            st.push(a);
        }
    }
}
```

```
static void reverse()
```

```
{
    if (st.size() > 0)
```

```
    {
        char x = st.peek();
        st.pop();
        reverse();
        insert_at_bottom (x);
    }
}
```

```
{
    public static void main (String args[])
    {
```

```
        st.push ('1');
        st.push ('2');
        st.push ('3');
        st.push ('4');
        syso ("Original Stack");
    }
```

```
syso(st);
```

```
reverse(c);
```

```
syso("reversed stack");
```

```
syso(st);
```

```
}
```

```
}
```